

# Тема на проект: Паралелно запълване на 2d таблица

Автор: Йоан Йорданов

## Технически детайли:

- 1) ОС: Linux, дистрибуция Debian на виртуална машина, 2 ядра, 2GB RAM
- 2) Език: C/C++

## Общ преглед

Реализира се клиент-сървър архитектура, която осъществява следната комуникация:

- 1) Клиентът израча на сървъра брой нишки и размери на двумерна таблица.
- 2) По получените данни от клиента сървърът създава съответния брой нишки и заделя памет за таблицата. След това нишките паралелно запълват таблицата съответно с различна стойност.
- 3) Сървърът изпраща таблицата на клиента и последният я отпечатва на екрана.

## Допълнителни функционалности

Сървърът отпечатва на екрана изминалото време за запълване на таблицата. За да се направи по-точно сравнение на работата на една нишка спрямо повече нишки, времето се засича непосредствено преди създаване и след приключване на нишките.

## Ключови детайли

- 1) Физическо представяне:

Двумерната таблица се представя в паметта като едномерен масив с дължина `size`, където:

```
int size = numRows * numColumns;
```

```
int d = size / numThreads; // брой елементи на нишка
```

```
int r = size % numThreads; //  $0 \leq r < numThreads$ 
```

Ако  $r \neq 0$ , то няма всяка нишка да има по равен брой клетки за запълване. За удобство е избрано последните  $r$  елемента да бъдат запълнени от основния процес.

- 2) Синхронизация:

Няма нужда от такава, понеже всяка нишка записва съответното си число от начален до краен индекс в масива, така че да не се припокриват интервалите на различните нишки. По-ясно ще стане от следната таблица:

| Индекс на нишка  | Начален индекс в масив | Краен индекс в масив |
|------------------|------------------------|----------------------|
| 0                | 0                      | $d - 1$              |
| 1                | $d$                    | $2 * d - 1$          |
| $i$              | $i * d$                | $(i + 1) * d - 1$    |
| $numThreads - 1$ | $(numThreads - 1) * d$ | $numThreads * d - 1$ |

Нишка  $i$  ( $0, 1, \dots, numThreads - 1$ ) записва числото  $i$  в интервала  $[i * d, (i + 1) * d - 1]$ .  
За яснота основни процес записва числото  $-1$  в  $[numThreads * d, numThreads * d + r - 1]$ .

### 3) Функция fillTable()

Тъй като не може да се определи в какъв момент след създаването си нишката започва да изпълнява функцията fillTable(), то е възможно след съответната итерация на цикъла нишката да започне изпълнението. Но тогава вече паметта на структурата Range ще бъде освободена. Затова е нужно да подадем на pthread\_create указател към динамично заделената структура Range, в противен случай след съответната итерация на цикъла паметта на структурата ще се освободи и нишката в функцията ще достъпи невалидна памет.

## Комуникация между клиент и сървър

### 1) Сървър

- създаване на TCP сокет  
`int sd = socket(AF_INET, SOCK_STREAM, 0);`
- промяна на опция на сокет  
`int optval = 1; // enabled`  
`int rv = setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));`  
да се използва локалният адрес/порт, който е бил използван от друг сокет.  
(да не се чака някакво време, за да се тества програмата).
- Присвоява на сокета локалния адрес на сървъра  
`bind(sd, (const struct sockaddr *) &addr, sizeof(addr))`
- Превръщане на сокет в пасивно състояние и добавяне на заявки за свързване от клиенти в опашка  
`int l = listen(sd, LISTEN_BACKLOG); // LISTEN_BACKLOG – максимален брой заявки в опашка`
- Създаване на нов сокет, като се взема първата заявка от опашката. Чрез файловия дескриптор, който връща accept, се извършва обмяна на байтове между клиент и сървър (read(), write()).  
`int ad = accept(sd, (struct sockaddr *) &addr, &addrlen);`

### 2) Клиент

- Създаване на TCP сокет  
`int sd = socket(AF_INET, SOCK_STREAM, 0);`
- Изпращане на заявка за свързване със сървър по даден адрес  
`int cd = connect(sd, (const struct sockaddr *) &addr, sizeof(addr));`

## Очаквани входни и изходни данни

### 1) numThreads = 1, numRows = 10, numCols = 10 (няма остатък)

```
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```

```

0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
0000000000

```

2) numThreads = 2, numRows = 10, numCols = 10 (няма остатък)

```

0000000000
0000000000
0000000000
0000000000
0000000000
0000000000
1111111111
1111111111
1111111111
1111111111
1111111111

```

3) numThreads = 6, numRows = 10, numCols = 10 (остатък 4)

```

6 10 10
0000000000
0000001111
1111111111
1122222222
2222222233
3333333333
3333444444
4444444444
5555555555
555555-1-1-1-1

```

Програмата е тествана за 1 нишка успешно до матрица с размери 28000 x 28000 и за 2 нишки до 25000 x 25000.

Резултати за една нишка: 12.5833, 11.7192, 11.0868 (секунди) (28000 x 28000)

Резултати за една нишка: 7.99996, 8.34726, 7.5929, 9.04814 (25000 x 25000)

Резултати за две нишки: 8.55606, 11.059, 8.12429, 7.66001 (25000 x 25000)

Резултати за една нишка: 0.00165, 0.00111, 0.00047, 0.00031 (100 x 100)

Резултати за две нишки: 0.00067, 0.000532, 0.000506, 0.000504

С оглед на резултатите (които разбира не са на базата на много тестове) не мога да открия значителна разлика между времето за 1 и 2 нишки. Възможно е една от причините е да броят на ядрата, дадени на виртуалната машина – 2. Ако има и други процеси, които операционната система на раздава на процесорите, то да доведе до по-бавно изпълнение. Или друга причина е времето за създаване на нишки и после тяхното изчакване да приключат.